

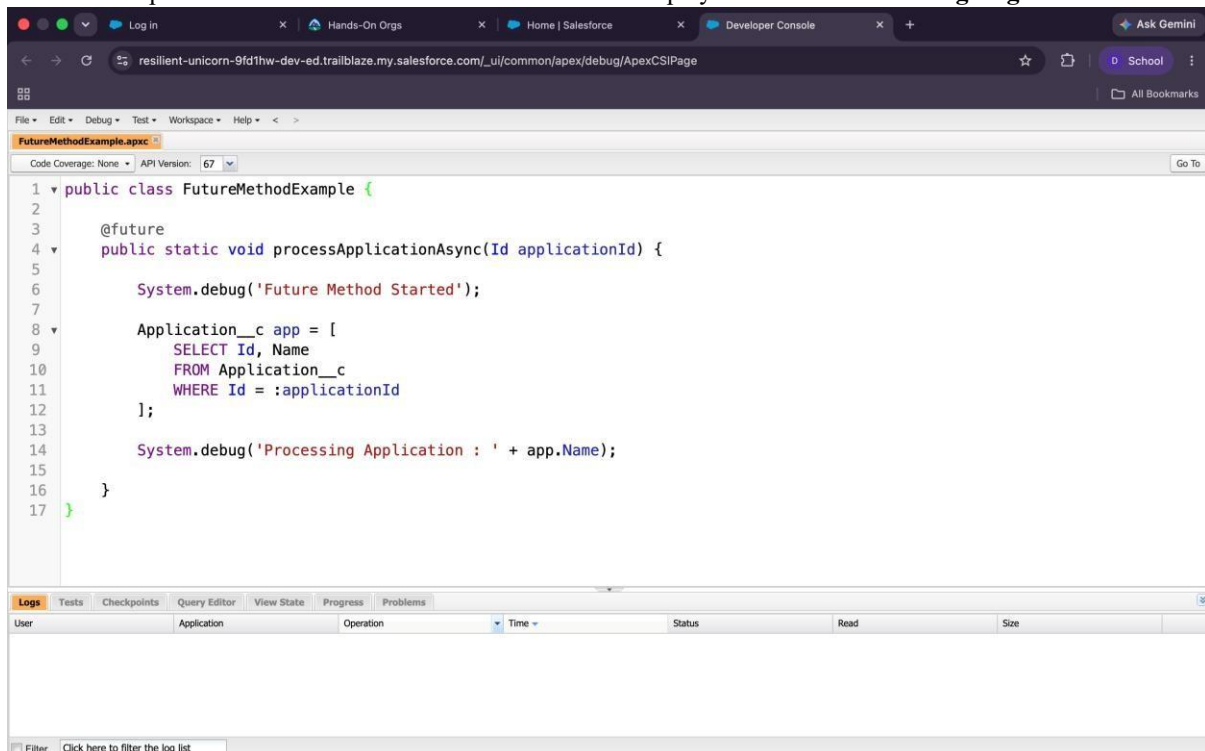
Experiment 1: Future Method Apex

Objective:

To implement a Future Method using the `@future` annotation to execute tasks asynchronously in the background, ensuring that time-consuming operations run separately without delaying the current user transaction.

Description:

Future Methods are used to execute time-consuming tasks asynchronously after the current transaction is completed. This improves system performance by allowing background processing. In this experiment, a Future Method is implemented to retrieve an **Account** record and display its details in the **Debug Log** for verification.



Apex Code :

```
public class FutureMethodExample {  
  
    @future    public static void processApplicationAsync(Id  
accountId) {  
  
        System.debug('Future Method Started');  
  
        Account acc = [  
  
            SELECT Id, Name  
  
            FROM Account  
  
            WHERE Id = :accountId  
  
        ];  
  
        System.debug('Processing Account: ' + acc.Name);  
    }  
}
```

```
}  
  
}
```

Experiment 2: Execute Future Method

Objective

To execute the Future Method using Execute Anonymous and verify that the asynchronous process starts successfully.

Description

The Future Method is invoked from the **Developer Console** using **Execute Anonymous**. The **Account ID** is provided as an input parameter, and Salesforce executes the method asynchronously in the background after the current transaction is completed. **Execute Anonymous**

Code :

```
FutureMethodExample.processApplicationAsync('001hg000007UBBhAAO');
```

The screenshot displays the Salesforce Developer Console interface. The top navigation bar includes tabs for 'Log in', 'Hands-On Orgs', 'Test Account | Account | Sales', and 'Developer Console'. The browser address bar shows the URL: `resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage`. The main workspace area is divided into two sections. The upper section, titled 'Execution Log', contains a table with columns for 'Timestamp', 'Event', and 'Details'. It lists various system events such as 'USER_INFO', 'EXECUTION_START', 'CODE_UNIT_STARTED', 'CODE_UNIT_FINISHED', 'HEAP_ALLOCATE', 'METHOD_ENTRY', and 'STATEMENT_EXECUTED'. The lower section, titled 'Logs', contains a table with columns for 'User', 'Application', 'Operation', 'Time', 'Status', 'Read', and 'Size'. It lists log entries for user 'DUDALA MADHAVI' with operations like `/services/data/v67.0/tooling/executeAnonymous` and `/services/data/v67.0/tooling/executeAnonymous`, all with a status of 'Success'.

Timestamp	Event	Details
10:28:22:001	USER_INFO	[EXTERNAL] 005hg000001en1R 23pa1a1258@resilient-unicorn-9fd1hw.com (GMT+05:30) India Standard Time (Asia/Kolkata) (GMT+05:30)
10:28:22:001	EXECUTION_START	
10:28:22:001	CODE_UNIT_STARTED	[EXTERNAL] FutureHandler - state load
10:28:22:004	CODE_UNIT_FINISHED	FutureHandler - state load
10:28:22:004	EXECUTION_FINISHED	
10:28:22:029	USER_INFO	[EXTERNAL] 005hg000001en1R 23pa1a1258@resilient-unicorn-9fd1hw.com (GMT+05:30) India Standard Time (Asia/Kolkata) (GMT+05:30)
10:28:22:029	EXECUTION_START	
10:28:22:029	CODE_UNIT_STARTED	[EXTERNAL] 01phg000001pLTF FutureMethodExample.processApplicationAsync(1d)
10:28:22:032	HEAP_ALLOCATE	[95] Bytes:3
10:28:22:032	HEAP_ALLOCATE	[100] Bytes:152
10:28:22:032	HEAP_ALLOCATE	[417] Bytes:408
10:28:22:032	HEAP_ALLOCATE	[430] Bytes:408
10:28:22:032	HEAP_ALLOCATE	[317] Bytes:6
10:28:22:032	HEAP_ALLOCATE	[EXTERNAL] Bytes:1
10:28:22:032	HEAP_ALLOCATE	[EXTERNAL] Bytes:4
10:28:22:032	METHOD_ENTRY	[1] 01phg000001pLTF FutureMethodExample.FutureMethodExample()
10:28:22:032	STATEMENT_EXECUTED	[1]
10:28:22:032	STATEMENT_EXECUTED	[1]
10:28:22:032	HEAP_ALLOCATE	[68] Bytes:5
10:28:22:032	HEAP_ALLOCATE	[74] Bytes:5
10:28:22:032	HEAP_ALLOCATE	[82] Bytes:7

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/executeAnonymous	07/08/2026, 10:28:22	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:28:22	Success	Unread	4.06 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/executeAnonymous	07/08/2026, 10:14:49	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:14:49	List has no rows for assignment to ...		4 KB

Experiment 3: Debug Log Verification

Objective:

To verify that the Future Method has started successfully using the Salesforce Debug Log.

Description:

The **Debug Log** displays the message "**Future Method Started**", confirming that the Future Method was successfully executed asynchronously by Salesforce.

The screenshot shows the Salesforce Developer Console interface. The top navigation bar includes tabs for 'Log in', 'Hands-On Orgs', 'Test Account | Account | Sale', and 'Developer Console'. The address bar shows the URL: `resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage`. The main area displays the 'Execution Log' for the 'FutureMethodExample.apxc' file. The log shows various system events, including 'SYSTEM_MODE...', 'METHOD_EXIT', 'HEAP_ALLOCATE', 'VARIABLE_SCO...', 'VARIABLE_ASSL...', 'STATEMENT_EX...', and 'HEAP_ALLOCATE'. A 'USER_DEBUG' log line is highlighted, showing the message '[6]DEBUG|Future Method Started'. A 'Log line #33 details' popup is open, displaying the log line: '[0:28:22:034 USER_DEBUG [6]DEBUG|Future Method Started]'. The bottom section shows the 'Logs' table with columns for 'User', 'Application', 'Operation', 'Time', 'Status', 'Read', and 'Size'.

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 10:28:22	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:28:22	Success	Unread	4.06 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 10:14:49	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:14:49	List has no rows for assignment to ...	Unread	4 KB

Experiment 4: Processing Account

Objective:

To verify that the Future Method successfully retrieved and processed the Account record.

Description:

The Debug Log displays the Account name that was processed by the Future Method, confirming successful execution.

The screenshot shows the Salesforce Developer Console interface. The top navigation bar includes tabs for 'Log in', 'Hands-On Orgs', 'Test Account | Account | Sale', and 'Developer Console'. The browser address bar shows the URL: `resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage`. The 'USER_DEBUG' tab is active, displaying the 'Execution Log'.

The 'Execution Log' table shows various events with timestamps, event names, and details. A popup window titled 'Log line #50 details' is open, showing the log line: `10:28:22:050 USER_DEBUG [14] | (DEBUG) Processing Account: Test Account`.

Below the log, there is a table with columns: User, Application, Operation, Time, Status, Read, and Size. The table contains four rows of log entries.

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute	07/08/2026, 10:28:22	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:28:22	Success	Unread	4.06 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute	07/08/2026, 10:14:49	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:14:49	List has no rows for assignment to ...		4 KB

Experiment 5: Queueable Apex

Objective:

To create a Queueable Apex class for structured background processing.

Description:

Queueable Apex enables asynchronous processing with greater flexibility and control than Future Methods. In this experiment, a **Queueable Apex** class processes **Account** records and displays debug messages to verify successful execution.

Apex Code :

```
public class AccountQueueableJob implements Queueable {

    public void execute(QueueableContext context) {

        System.debug('Queueable Job Started');

        List<Account> accList = [

            SELECT Id, Name

            FROM Account

        ];

        for(Account acc : accList){

            System.debug('Processing Account: ' + acc.Name);

        }

    }

}
```

```

        System.debug('Queueable Job Completed');
    }
}

```

The screenshot shows the Salesforce Developer Console with the `AccountQueueableJob.apxc` class open. The class implements the `Queueable` interface and contains an `execute` method. The method performs a SOQL query to retrieve account IDs and names, then iterates through the results, logging each account name. Finally, it logs a completion message.

```

1 public class AccountQueueableJob implements Queueable {
2
3     public void execute(QueueableContext context) {
4
5         System.debug('Queueable Job Started');
6
7         List<Account> acclist = [
8             SELECT Id, Name
9             FROM Account
10        ];
11
12        for(Account acc : acclist){
13            System.debug('Processing Account: ' + acc.Name);
14        }
15
16        System.debug('Queueable Job Completed');
17    }
18 }

```

Below the code editor, the **Logs** tab is active, displaying a table of execution logs:

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 10:28:22	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:28:22	Success	Unread	4.06 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 10:14:49	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:14:49	List has no rows for assignment to ...		4 KB

At the bottom of the logs panel, there is a filter button and a link to filter the log list.

Experiment 6: Execute Queueable Apex

Objective:

To execute the Queueable Apex job asynchronously.

Execute Anonymous Code :

```
System.enqueueJob(new AccountQueueableJob());
```

The screenshot displays the Salesforce Developer Console interface. The top navigation bar includes tabs for 'Log in', 'Hands-On Orgs', 'Test Account | Account | Sales', and 'Developer Console'. The address bar shows the URL: `resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage`. The main content area is divided into two sections: an 'Execution Log' and a 'Log list'.

Execution Log: This section shows a detailed log of events for the Apex job. The log is organized into columns: 'Timestamp', 'Event', and 'Details'. The events include 'USER_INFO', 'EXECUTION_START', 'CODE_UNIT_STARTED', 'HEAP_ALLOCATE', 'STATEMENT_EXECUTED', 'METHOD_ENTRY', and 'HEAP_ALLOCATE'. The details column provides specific information for each event, such as the user ID, the code unit name, and the heap allocation details.

Log list: This section provides a summary of the log entries. It includes columns for 'User', 'Application', 'Operation', 'Time', 'Status', 'Read', and 'Size'. The log list shows several entries, including 'QueueableHandler' and 'FutureHandler', with their respective execution times and statuses.

Experiment 7: Queueable Debug Messages

Objective:

To verify successful execution of the Queueable Apex job using Debug Logs.

Description:

- **Queueable Job Started** – Job execution started.
- **Processing Account** – Account record processed.
- **Queueable Job Completed** – Job executed successfully.

resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

USER_DEBUG 1/3

AccountQueueableJob.apxc Log executeAnonymous @07/08/2026, 11:19:51 Log executeAnonymous @07/08/2026, 11:21:17

Execution Log

Timestamp	Event	Details
11:21:17:020	VARIABLE_ASSI...	[12][acc{"Id":"001hg0000068VYAA0","Name":"Sample Account for E (11 more ...)"}](0x7b8126d0
11:21:17:020	STATEMENT_EX...	[12]
11:21:17:020	STATEMENT_EX...	[13]
11:21:17:020	HEAP_ALLOCATE	[13] Bytes:51
11:21:17:020	USER_DEBUG	[13] DEBUG Processing Account: Sample Account for Entitlements
11:21:17:020	HEAP_ALLOCATE	[12] Bytes:5
11:21:17:020	HEAP_ALLOCATE	[12] Bytes:12
11:21:17:020	VARIABLE_SCO...	[12][acc Account true false
11:21:17:020	VARIABLE_ASSI...	[12][acc{"Id":"001hg000007U8BHAAD","Name":"Test Account"}](0x532f286
11:21:17:020	STATEMENT_EX...	[12]
11:21:17:020	STATEMENT_EX...	[13]
11:21:17:020	HEAP_ALLOCATE	[13] Bytes:32
11:21:17:020	USER_DEBUG	[13] DEBUG Processing Account: Test Account
11:21:17:020	HEAP_ALLOCATE	[12] Bytes:5
11:21:17:020	VARIABLE_ASSI...	[12][acc null
11:21:17:020	STATEMENT_EX...	[16]
11:21:17:020	HEAP_ALLOCATE	[16] Bytes:23
11:21:17:020	USER_DEBUG	[16] DEBUG Queueable Job Completed
11:21:17:020	SYSTEM_MODE...	false
11:21:17:020	CODE_UNIT_FL...	AccountQueueableJob
11:21:17:020	EXECUTION_FL...	

Log line #169 details

11:21:17:020 USER_DEBUG [16]|DEBUG|Queueable Job Completed

OK

Logs Tests Checkpoints Query Editor View State Progress Problems

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	QueueableHandler	07/08/2026, 11:21:17	Success		10.82 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execut...	07/08/2026, 11:21:16	Success	Unread	2.93 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execut...	07/08/2026, 11:19:51	Success	Unread	2.95 KB
DUDALA MADHAVI	Unknown	QueueableHandler	07/08/2026, 11:19:51	Success	Unread	10.8 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execut...	07/08/2026, 10:28:22	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:28:22	Success		4.06 KB

Filter Click here to filter the log list

Experiment 8: Batch Apex

Objective:

To create a Batch Apex class that processes Account records in batches.

Description:

Batch Apex is used to process large volumes of records asynchronously by dividing them into smaller batches. In this experiment, a **Batch Apex** class is implemented using the **start()**, **execute()**, and **finish()** methods to process records efficient.

resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

Developer Console

AccountBatchJob.apxc Log executeAnonymous @07/08/2026, 11:24:49

Execution Log

Timestamp	Event	Details
11:24:49:000	USER_INFO	[EXTERNAL][005hg000001en1R123pa1a1258@resilient-unicorn-9fd1hw.com](GMT+05:30) India Standard Time (Asia/Kolkata) GMT+05:30
11:24:49:000	EXECUTION_ST...	
11:24:49:000	CODE_UNIT_ST...	[EXTERNAL][01phg000001pbv1 AccountBatchJob
11:24:49:003	HEAP_ALLOCATE	[EXTERNAL] Bytes:4
11:24:49:015	HEAP_ALLOCATE	[95] Bytes:3
11:24:49:015	HEAP_ALLOCATE	[100] Bytes:152
11:24:49:015	HEAP_ALLOCATE	[417] Bytes:408
11:24:49:015	HEAP_ALLOCATE	[430] Bytes:408
11:24:49:015	HEAP_ALLOCATE	[317] Bytes:6
11:24:49:015	HEAP_ALLOCATE	[EXTERNAL] Bytes:11
11:24:49:015	SYSTEM_METH...	[15] BatchableContextImpl.BatchableContextImpl()
11:24:49:015	STATEMENT_EX...	[15]
11:24:49:015	SYSTEM_METH...	[15] BatchableContextImpl
11:24:49:015	HEAP_ALLOCATE	[EXTERNAL] Bytes:12
11:24:49:015	HEAP_ALLOCATE	[EXTERNAL] Bytes:4
11:24:49:015	VARIABLE_SCO...	[31] this Database.BatchableContextImpl true false
11:24:49:015	VARIABLE_ASSI...	[31] this () 0x4176965
11:24:49:015	VARIABLE_SCO...	[31] jobId Id false false
11:24:49:015	VARIABLE_ASSI...	[31] jobId 707hg000001jySAAE"
11:24:49:015	VARIABLE_SCO...	[31] childJobId Id false false
11:24:49:015	VARIABLE_ASSI...	[31] childJobId null

Logs Tests Checkpoints Query Editor View State Progress Problems

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:24:50	Success	Unread	3.7 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:24:50	Success	Unread	5.52 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:24:50	Success	Unread	4.78 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execut...	07/08/2026, 11:24:49	Success	Unread	2.98 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:24:49	Success		3.83 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:24:49	Success	Unread	2.53 KB

Filter Click here to filter the log list

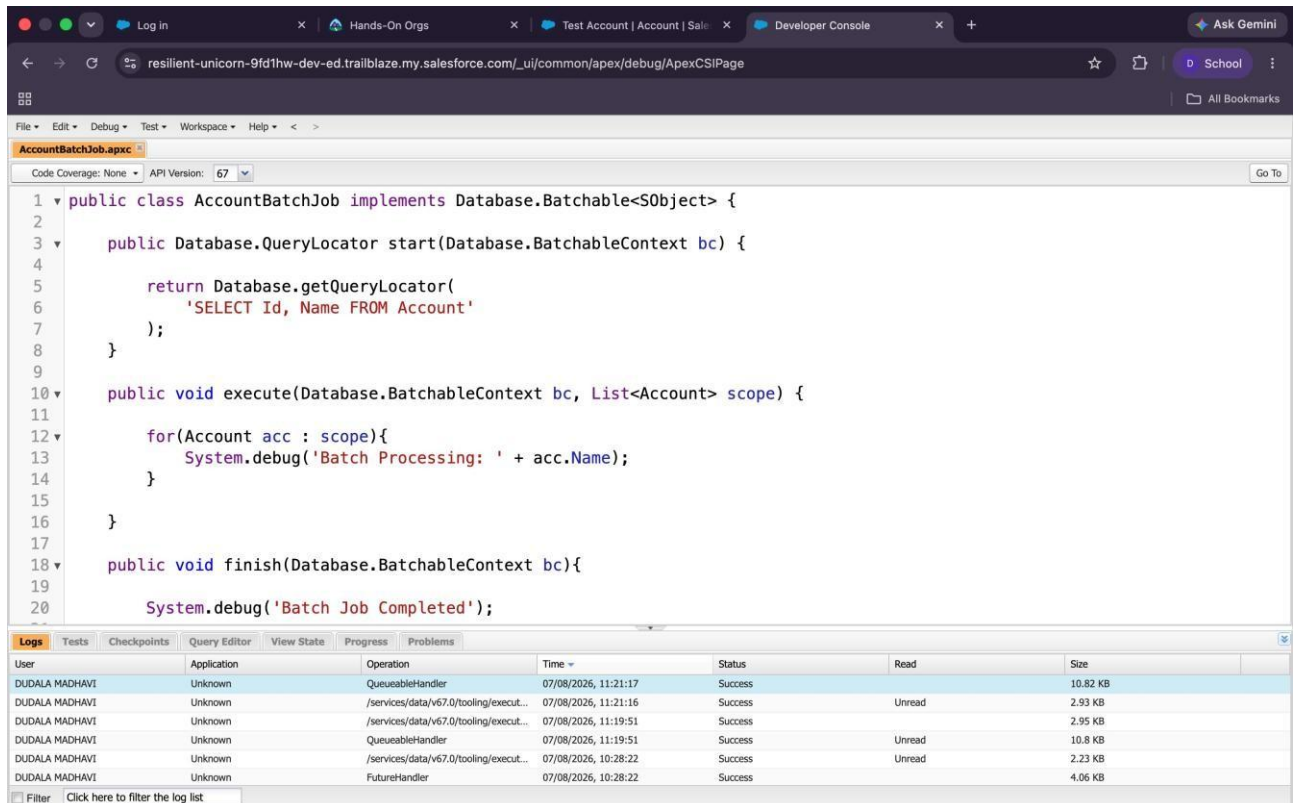
Experiment 9: Batch Execution

Objective:

To execute the Batch Apex class using Execute Anonymous.

Execute Anonymous Code :

```
Database.executeBatch(new AccountBatchJob(), 5);
```



The screenshot displays the Salesforce Developer Console interface. The top section shows the code editor for the `AccountBatchJob.apxc` class, which implements the `Database.Batchable<SObject>` interface. The code includes methods for `start`, `execute`, and `finish`. The `execute` method iterates over a list of accounts and logs the batch processing status. The bottom section shows the `Logs` tab, which displays a list of log entries for the user `DUDALA MADHAVI`. The logs indicate successful execution of the batch job, with the status `Success` and the message `Batch Job Completed`.

```
1 public class AccountBatchJob implements Database.Batchable<SObject> {
2
3     public Database.QueryLocator start(Database.BatchableContext bc) {
4
5         return Database.getQueryLocator(
6             'SELECT Id, Name FROM Account'
7         );
8     }
9
10    public void execute(Database.BatchableContext bc, List<Account> scope) {
11
12        for(Account acc : scope){
13            System.debug('Batch Processing: ' + acc.Name);
14        }
15    }
16
17    public void finish(Database.BatchableContext bc){
18
19        System.debug('Batch Job Completed');
20    }
21 }
```

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	QueueableHandler	07/08/2026, 11:21:17	Success		10.82 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 11:21:16	Success	Unread	2.93 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 11:19:51	Success	Unread	2.95 KB
DUDALA MADHAVI	Unknown	QueueableHandler	07/08/2026, 11:19:51	Success	Unread	10.8 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute...	07/08/2026, 10:28:22	Success	Unread	2.23 KB
DUDALA MADHAVI	Unknown	FutureHandler	07/08/2026, 10:28:22	Success		4.06 KB

Experiment 10: Batch Debug Messages

Objective:

To verify successful execution of the Batch Apex job using Debug Logs.

Description:

The Debug Log displays "Batch Job Completed", confirming successful batch execution.

Execution Log

Timestamp	Event	Details
11:26:56:027	STATEMENT_EX...	[20]
11:26:56:027	HEAP_ALLOCATE	[20]Bytes:19
11:26:56:027	USER_DEBUG	[20][DEBUG]Batch Job Completed
11:26:56:027	SYSTEM_MODE...	false
11:26:56:051	CUMULATIVE_L...	
11:26:56:051	LIMIT_USAGE...	(default)
11:26:56:000	LIMIT_USAGE...	Number of SOQL queries: 0 out of 200
11:26:56:000	LIMIT_USAGE...	Number of query rows: 0 out of 50000
11:26:56:000	LIMIT_USAGE...	Number of SOSL queries: 0 out of 20
11:26:56:000	LIMIT_USAGE...	Number of DML statements: 0 out of 150
11:26:56:000	LIMIT_USAGE...	Number of Publish Immediate DML: 0 out of 150
11:26:56:000	LIMIT_USAGE...	Number of DML rows: 0 out of 10000
11:26:56:000	LIMIT_USAGE...	Maximum CPU time: 0 out of 60000
11:26:56:000	LIMIT_USAGE...	Maximum heap size: 0 out of 12000000
11:26:56:000	LIMIT_USAGE...	Number of callouts: 0 out of 0
11:26:56:000	LIMIT_USAGE...	Number of Email Invocations: 0 out of 10
11:26:56:000	LIMIT_USAGE...	Number of future calls: 0 out of 0
11:26:56:000	LIMIT_USAGE...	Number of queueable jobs added to the queue: 0 out of 1
11:26:56:000	LIMIT_USAGE...	Number of Mobile Apex push calls: 0 out of 10

Log line #45 details

11:26:56:027 USER_DEBUG [20][DEBUG]Batch Job Completed

OK

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execut...	07/08/2026, 11:26:56	Success	Unread	2.95 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:26:56	Success	Unread	3.7 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:26:56	Success	Unread	3.83 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	2.53 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	4.77 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	5.51 KB

Experiment 11: Scheduled Apex

Objective :

To create a Scheduled Apex class that executes automatically at a specified time.

Description:

Scheduled Apex automates background processing based on a defined schedule using the Schedulable interface.

```

1 public class AccountScheduler implements Schedulable {
2
3     public void execute(SchedulableContext sc) {
4
5         System.debug('Scheduled Job Started');
6
7         List<Account> accounts = [
8             SELECT Id, Name
9             FROM Account
10        ];
11
12        for(Account acc : accounts){
13            System.debug('Scheduled Processing: ' + acc.Name);
14        }
15
16        System.debug('Scheduled Job Completed');
17    }
18 }

```

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execut...	07/08/2026, 11:26:56	Success	Unread	2.95 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:26:56	Success	Unread	3.7 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:26:56	Success	Unread	3.83 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	2.53 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	4.77 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	5.51 KB

Experiment 12: Schedule Apex Job

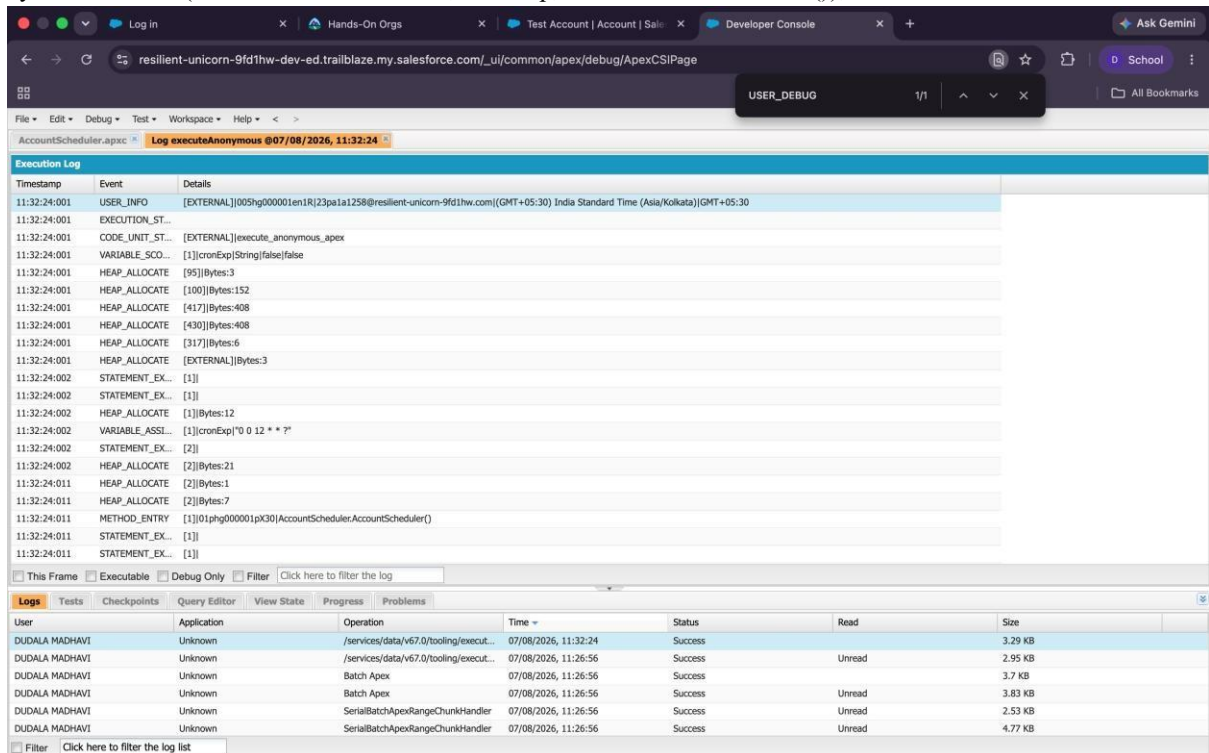
Objective:

To schedule the Apex class using a CRON expression.

Execute Anonymous Code :

String cronExp = '0 0 12 * * ?';

```
System.schedule('Account Scheduler Job', cronExp, new AccountScheduler());
```



The screenshot displays the Salesforce Developer Console interface. The top navigation bar includes tabs for 'Log in', 'Hands-On Orgs', 'Test Account | Account | Sales', and 'Developer Console'. The browser address bar shows the URL: `resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage`. The main content area is titled 'AccountScheduler.apxc' and shows a log for an anonymous execution at 11:32:24 on 07/08/2026. The log is organized into columns: Timestamp, Event, and Details. The events include USER_INFO, EXECUTION_START, CODE_UNIT_START, VARIABLE_SCOPE, HEAP_ALLOCATE, STATEMENT_EXECUTE, and METHOD_ENTRY. The details for the STATEMENT_EXECUTE event show the CRON expression: `'0 0 12 * * ?'`. Below the log, there is a 'Logs' tab with a table of execution results.

User	Application	Operation	Time	Status	Read	Size
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute/anonymous	07/08/2026, 11:32:24	Success		3.29 KB
DUDALA MADHAVI	Unknown	/services/data/v67.0/tooling/execute/anonymous	07/08/2026, 11:26:56	Success	Unread	2.95 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:26:56	Success	Unread	3.7 KB
DUDALA MADHAVI	Unknown	Batch Apex	07/08/2026, 11:26:56	Success	Unread	3.83 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	2.53 KB
DUDALA MADHAVI	Unknown	SerialBatchApexRangeChunkHandler	07/08/2026, 11:26:56	Success	Unread	4.77 KB

Experiment 13: Verify Scheduled Job

Objective:

To verify that the Scheduled Apex job has been successfully created.

Description:

The **Scheduled Jobs** page displays the scheduled job name, next execution time, and job type, confirming that the Apex job has been scheduled successfully.

The screenshot shows the Salesforce Setup page for Scheduled Jobs. The browser address bar indicates the URL: `resilient-unicorn-9fd1hw-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/ScheduledJobs/home`. The page title is "Scheduled Jobs".

On the left sidebar, the "Setup" menu is open, showing "Home" and "Object Manager". The "Scheduled Jobs" link is highlighted under the "Jobs" section.

The main content area is titled "All Scheduled Jobs". It includes a message: "The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so."

A yellow banner indicates the "Percentage of Scheduled Jobs Used: 1%". It states: "You have currently used 1 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Apex Limits](#) topic."

Below the banner, there is a "View:" dropdown set to "All Scheduled Jobs" and a "Create New View" link.

A table displays the scheduled jobs. The table has columns: Action, Job Name, Submitted By, Submitted, Started, Next Scheduled Run, Type, and Cron Trigger ID. The table is filtered by "Schedule Apex".

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Account Scheduler Job	MADHAVI DUDALA	07/08/2026, 11:32 am		07/08/2026, 12:00 pm	Scheduled Apex	08ehg000000ghwx
Del	Metalytics Data Loader Job for Org : 00Dhg000002hb3B	User Integration	28/07/2026, 10:06 am	07/08/2026, 6:44 am	08/08/2026, 6:44 am	Autonomous Data Loader Job	08ehg000000cGLW

At the bottom of the table, there is a filter bar with letters A through Z and an "All" button.